VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

BUKKAPATNAM VARSHYARA (1BM24CS075)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"ANALYSIS AND DESIGN OF ALGORITHMS"** carried out by Bukkapatnam Varshyara **(1BM24CS075)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Pallavi C V
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Lab program 1:

Code:

```c
#include <stdio.h>
#include <time.h>
#define MAX 100
int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int n;
void dfs(int v)
{
    visited[v] = 1;
    for (int i = 0; i < n; i++)
    {
        if (graph[v][i] && !visited[i])
        {
            dfs(i);
        }
    }
    stack[++top] = v;
}
void topologicalSort()
{
    for (int i = 0; i < n; i++)
    {
        if (!visited[i])
        {
            dfs(i);
        }
    }
    printf("\nTopological Ordering:\n");
    while (top != -1)
    {
        printf("%d ", stack[top--]);
    }
    printf("\n");
}
```

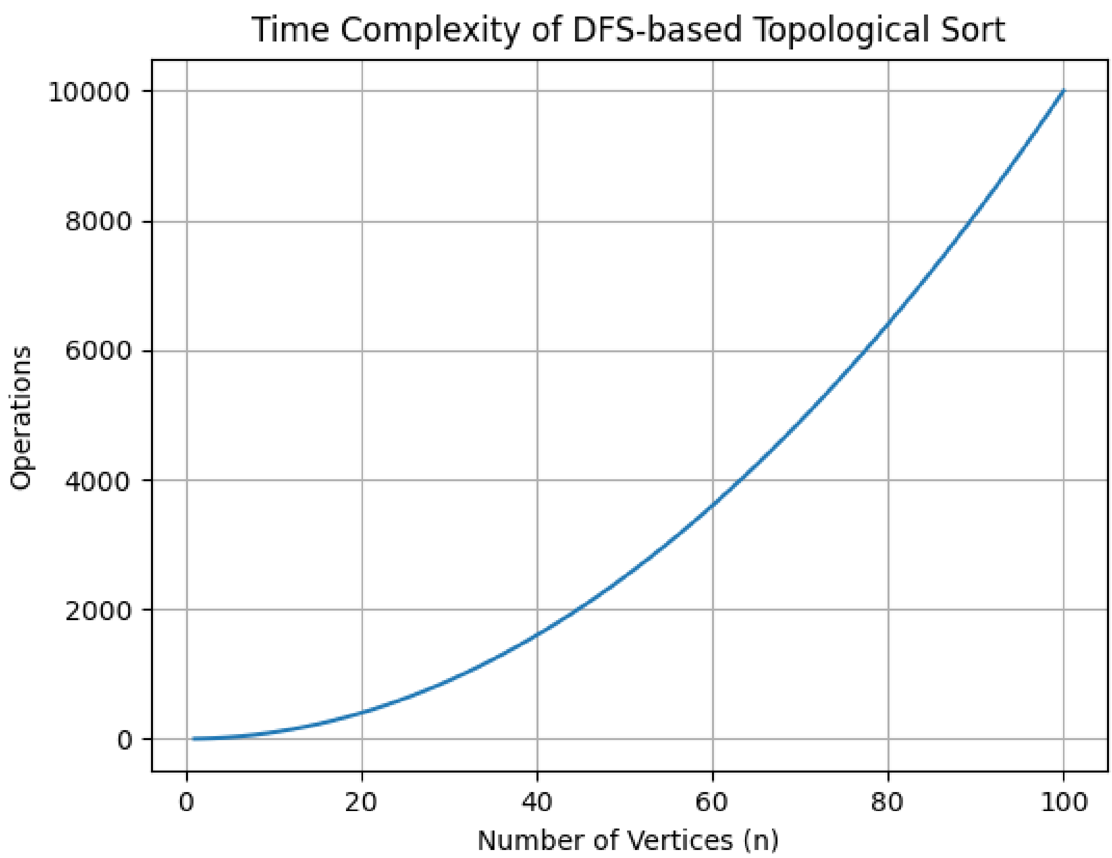
```c
int main()
{
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            scanf("%d", &graph[i][j]);
        }
    }
    for (int i = 0; i < n; i++)
    {
        visited[i] = 0;
    }
    start = clock();
    topologicalSort();
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nExecution Time = %lf seconds\n", cpu_time_used);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Topological Ordering:
0 2 1 3

Execution Time = 0.002000 seconds
```

Graph:

Time Complexity : O(n^2)



Lab Program 2:

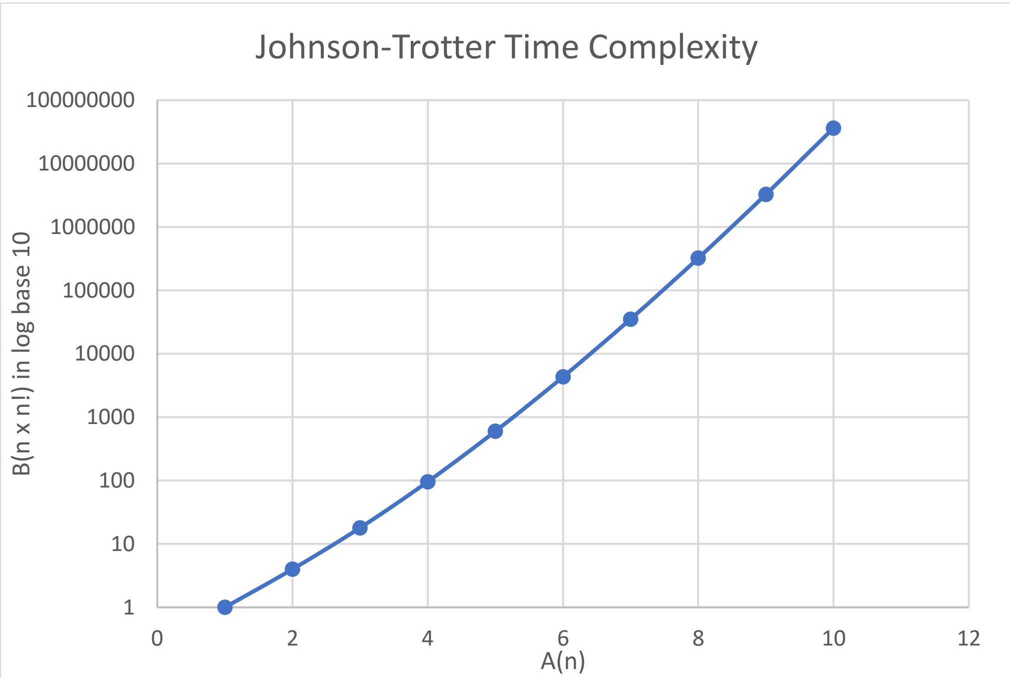
```c
#include <stdio.h>
#include <time.h>
int NN;
int p[100], pi[100];
int dir[100];
void PrintPerm()
{
    for (int i = 1; i <= NN; i++)
        printf("%d ", p[i int NN
    printf("\n");
}
void Move(int x, int d)
{
    int z;
    z = p[pi[x] + d];
    p[pi[x]] = z;
    p[pi[x] + d] = x;
    pi[z] = pi[x];
    pi[x] = pi[x] + d;
}
void Perm(int n)
{
    if (n > NN)
        PrintPerm();
    else
    {
        Perm(n + 1);
        for (int i = 1; i <= n - 1; i++)
        {
            Move(n, dir[n]);
            Perm(n + 1);
        }
        dir[n] = -dir[n];
    }
}
int main()
{
    clock_t start, end;
    double execution_time;
    printf("Enter n: ");
    scanf("%d", &NN);
    for (int i = 1; i <= NN; i++)
    {
        p[i] = i;
        pi[i] = i;
        dir[i] = -1;
    }
    printf("\nPermutations:\n");
    start = clock();
    Perm(1);
    end = clock();
    execution_time =
        (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nExecution Time = %lf seconds\n",
        execution_time);
    return 0;
}
```

Output:

```
Enter n: 3

Permutations:
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Execution Time = 0.003000 seconds
```

Graph:

Time Complexity : O(n x n!)



Lab Program 3:

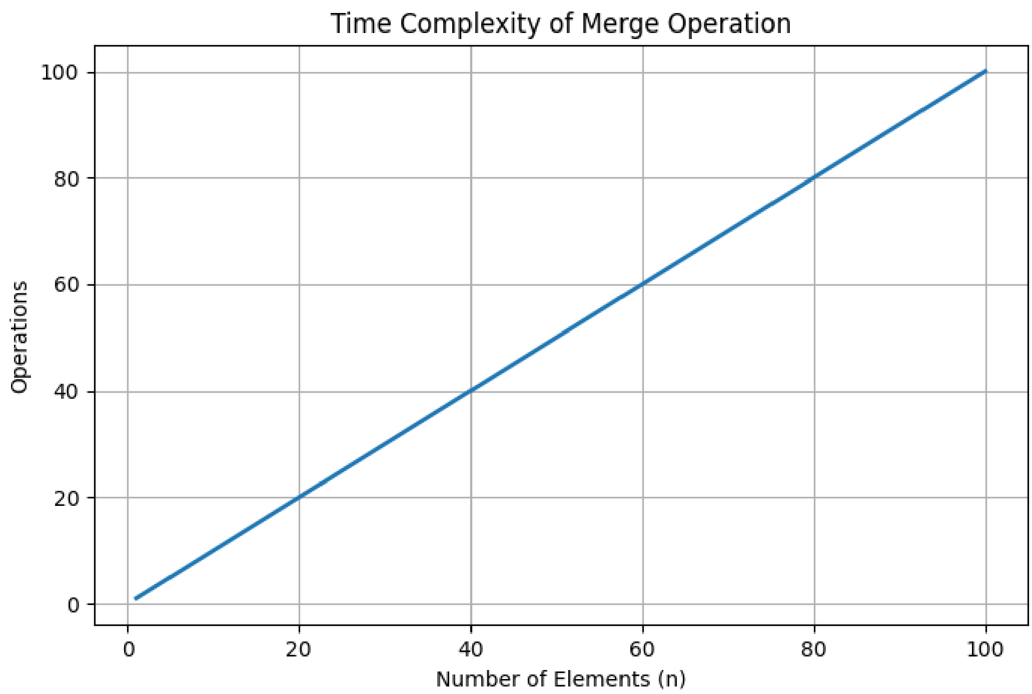
```c
#include <stdio.h>
#include <time.h>
#define MAX 100
int main()
{
    int low, mid, high;
    int a[MAX], b[MAX];
    printf("Enter low index: ");
    scanf("%d", &low);
    printf("Enter mid index: ");
    scanf("%d", &mid);
    printf("Enter high index: ");
    scanf("%d", &high);
    printf("\nEnter %d sorted elements:\n", high + 1);
    for (int i = low; i <= high; i++)
        scanf("%d", &a[i]);
    clock_t start, end;
    double execution_time;
    start = clock();
    int i = low;
    int j = mid + 1;
    int k = low;
    while (i <= mid && j <= high)
    {
        if (a[i] <= a[j])
        {
            b[k] = a[i];
            i++;
        }
        else
        {
            b[k] = a[j];
            j++;
        }
        k++;
    }

    while (i <= mid)
    {
        b[k] = a[i];
        i++;
        k++;
    }
    while (j <= high)
    {
        b[k] = a[j];
        j++;
        k++;
    }
    end = clock();
    execution_time =
        ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nMerged Array:\n");
    for (k = low; k <= high; k++)
        printf("%d ", b[k]);
    printf("\n");
    printf("\nExecution Time = %lf seconds\n",
           execution_time);
    return 0;
}
```

Output:

```
Enter low index: 0
Enter mid index: 3
Enter high index: 7

Enter 8 sorted elements:
10 20 30 40 15 25 35 45

Merged Array:
10 15 20 25 30 35 40 45

Execution Time = 0.000000 seconds
```

Graph:

Time Complexitu : O(n)



Lab Program 4:

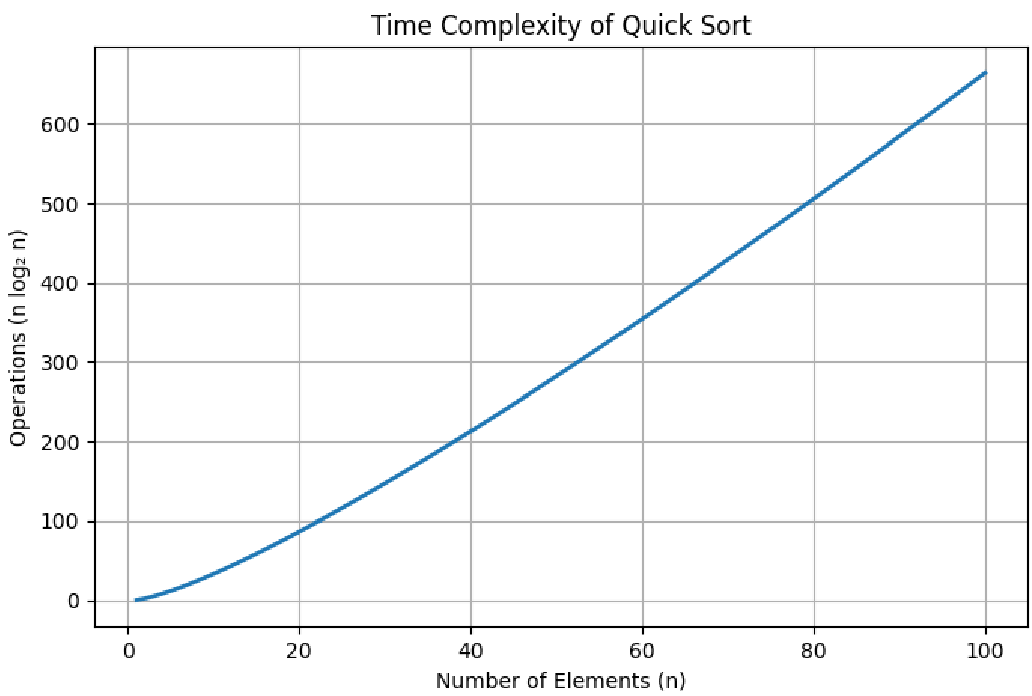
```c
#include <stdio.h>
#include <time.h>
#define MAX 100
int partition(int a[], int low, int high)
{
    int pivot = a[low];
    int i = low + 1;
    int j = high;
    int temp;
    while (1)
    {
        while (i <= high && a[i] <= pivot)
            i++;
        while (a[j] > pivot)
            j--;
        if (i < j)
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
        else
        {
            temp = a[low];
            a[low] = a[j];
            a[j] = temp;
            return j;
        }
    }
}
void quicksort(int a[], int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition(a, low, high);
        quicksort(a, low, pivotIndex - 1);
        quicksort(a, pivotIndex + 1, high);
    }
}
int main()
{
    int a[MAX];
    int n;
    clock_t start, end;
    double execution_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    quicksort(a, 0, n - 1);
    end = clock();
    execution_time =
        (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nSorted Array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
    printf("\nExecution Time = %lf seconds\n",
           execution_time);
    return 0;
}
```

Output:

```
Enter number of elements: 10
Enter elements:
78 3 44 16 9 33 28 14 99 88

Sorted Array:
3 9 14 16 28 33 44 78 88 99

Execution Time = 0.000000 seconds
```

Graph:

Time complexity : O(n log n)



Lab Program 5:

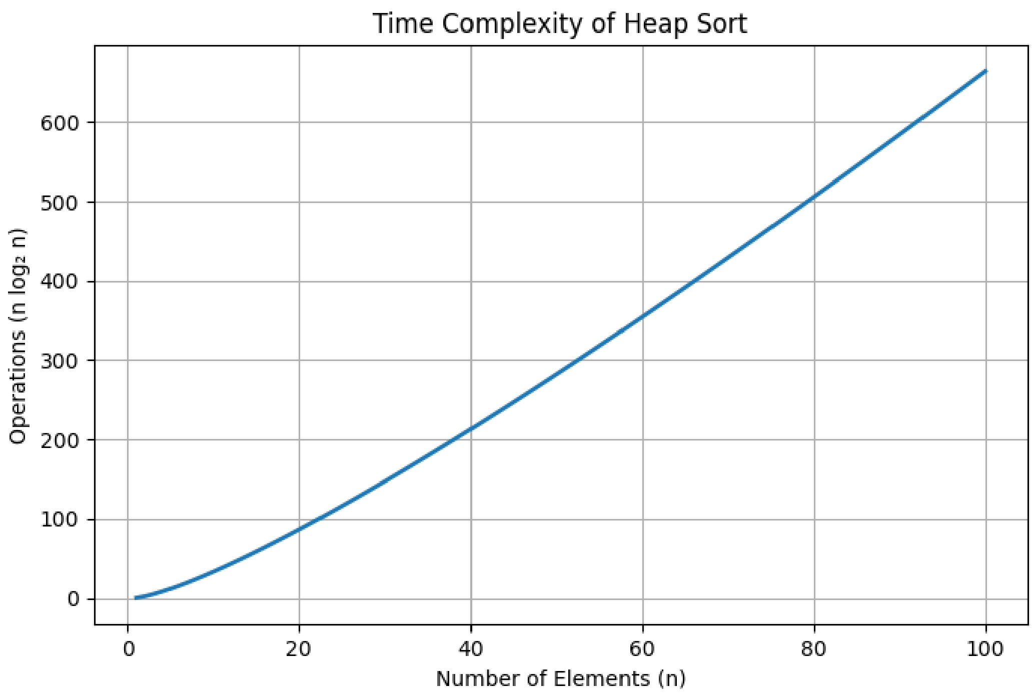
```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void heapify(int a[], int n, int i)
{
    int c, item;
    int p=i;
    item=a[p];
    c=2*p+1;
    while(c<=n-1){
        if(c+1<=n-1){
            if(a[c]<a[c+1]) c++;
        }
        if(item<a[c]){
            a[p]=a[c];
            p=c;
            c=2*p+1;
        }
        else break;
    }
    a[p]=item;
}
void buildheap(int a[],int n){
    int i;
    for(i=(n-1)/2;i>=0;i--){
        heapify(a,n,i);
    }
}
void heapSort(int a[], int n)
{
    int i, temp;
    buildheap(a,n);
    for(i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;
        heapify(a,i,0);
    }
}
int main()
{
    int n;
    clock_t start, end;
    double cpu_time;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    start = clock();
    heapSort(a, n);
    end = clock();
    cpu_time = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nSorted elements are:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n\nTime taken = %f seconds\n", cpu_time);
    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements:
4 2 6 1 8 3

Sorted elements are:
1 2 3 4 6 8

Time taken = 0.000000 seconds
```

Graph:

Time Complexity : O(n log n)



Lab Program 6:

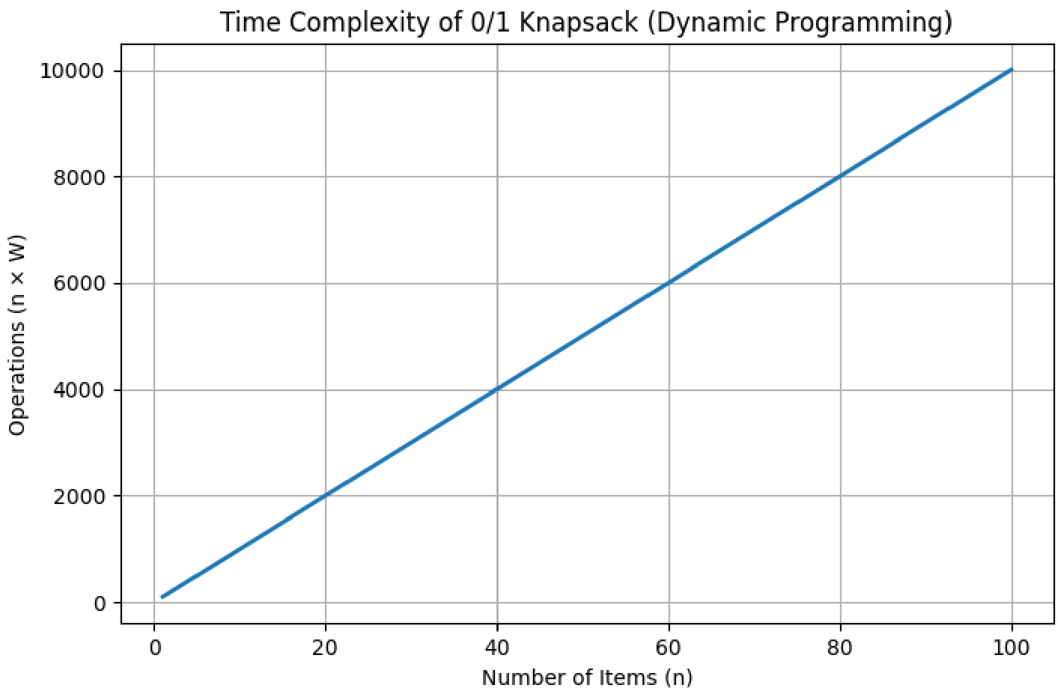
```c
#include <stdio.h>
#include <time.h>
int max(int a, int b) {
    return (a > b) ? a : b;
}
// 0/1 Knapsack using Dynamic Programming
int knapsack(int W, int wt[], int pt[], int n) {
    int i, w;
    int K[n + 1][W + 1];
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                K[i][w] = 0;
            else if (wt[i - 1] <= w)
                K[i][w] = max(pt[i - 1] + K[i - 1][w - wt[i - 1]],
                              K[i - 1][w]);
            else
                K[i][w] = K[i - 1][w];
        }
    }
    return K[n][W];
}
int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int pt[n], wt[n];
    printf("Enter weights of items:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &wt[i]);
    printf("Enter profit of items:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &pt[i]);
    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    clock_t start, end;
    start = clock();    // Start time
    int result = knapsack(W, wt, pt, n);
    end = clock();      // End time
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nMaximum Profit = %d\n", result);
    printf("Execution Time = %f seconds\n", time_taken);
    return 0;
}
```

Output:

```
Enter number of items: 3
Enter weights of items:
2 3 5
Enter profit of items:
20 10 40
Enter capacity of knapsack: 6

Maximum Profit = 40
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : O (nW)



Lab Program 7:

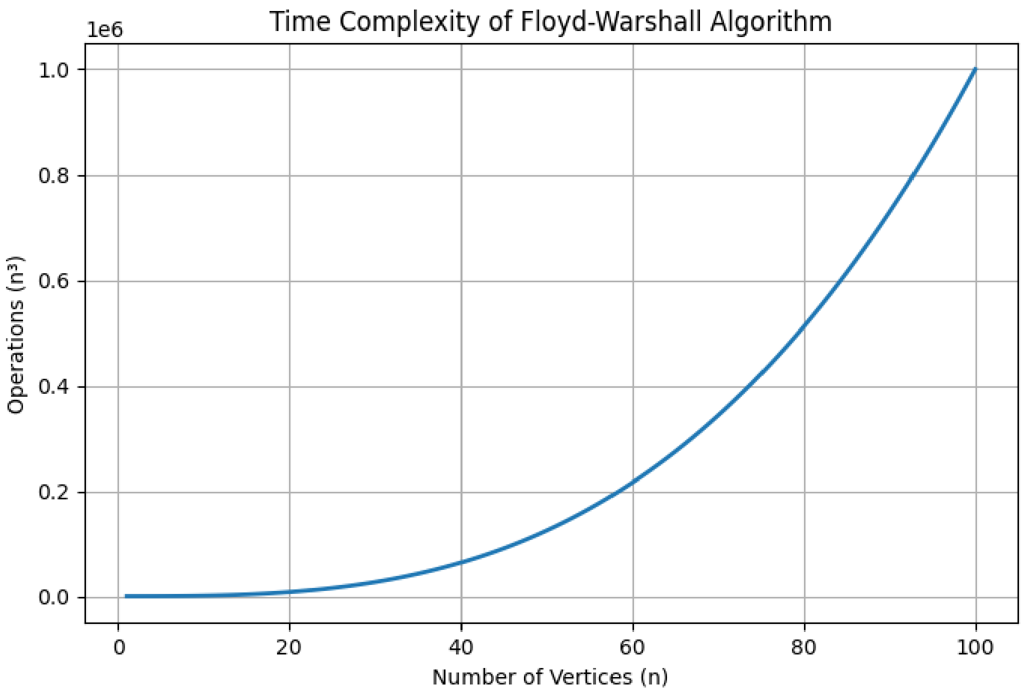
```c
#include <stdio.h>
#include <time.h>
#define MAX 20
#define INF 9999
void floyds(int n, int cost[MAX][MAX], int A[MAX][MAX]) {
    int i, j, k;
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            A[i][j] = cost[i][j];
        }
    }
    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(A[i][k] + A[k][j] < A[i][j]) {
                    A[i][j] = A[i][k] + A[k][j];
                }
            }
        }
    }
}
int main() {
    int n;
    int cost[MAX][MAX], A[MAX][MAX];
    int i, j;
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter the cost matrix:\n");
    printf("(Enter %d for INF / no direct edge)\n", INF);
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
    start = clock();
    floyds(n, cost, A);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nShortest Distance Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            if(A[i][j] == INF)
                printf("INF\t");
            else
                printf("%d\t", A[i][j]);
        }
        printf("\n");
    }
    printf("\nExecution Time: %f seconds\n", cpu_time_used);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter the cost matrix:
(Enter 9999 for INF / no direct edge)
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0

Shortest Distance Matrix:
0       5       8       9
INF     0       3       4
INF     INF     0       1
INF     INF     INF     0

Execution Time: 0.000000 seconds
```

Graph:

Time Complexity : O(n^3)



Lab Program 8:

a)

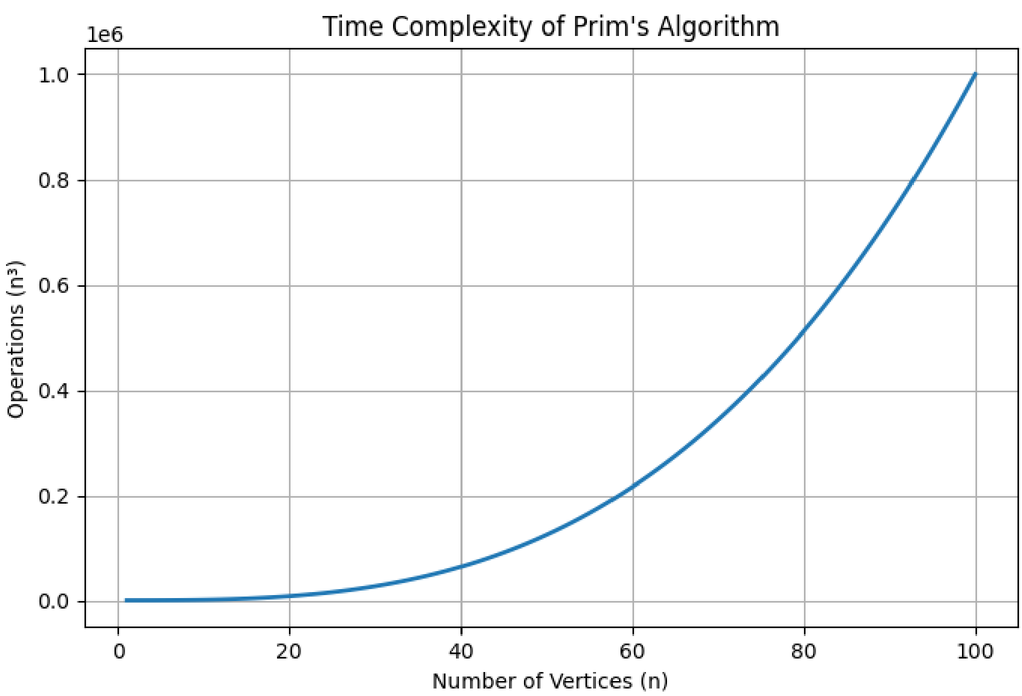
```c
#include <stdio.h>
#include <time.h>
#define MAX 20
#define INF 9999
int main()
{
    int n;
    int cost[MAX][MAX];
    int visited[MAX] = {0};
    int minCost = 0;
    int edges = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter Cost Adjacency Matrix:\n");
    printf("(Enter %d for no edge)\n", INF);
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);
    }
    clock_t start, end;
    double execution_time;
    start = clock();
    visited[0] = 1;
    printf("\nEdges in MST:\n");
    while(edges < n - 1)
    {
        int min = INF;
        int u = -1, v = -1;
        for(int i = 0; i < n; i++)
        {
            if(visited[i])
            {
                for(int j = 0; j < n; j++)
                {
                    if(!visited[j] && cost[i][j] < min)
                    {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }
        printf("%d -> %d  Cost = %d\n", u, v, min);
        minCost += min;
        visited[v] = 1;
        edges++;
    }
    end = clock();
    execution_time =(double)(end - start) / CLOCKS_PER_SEC;
    printf("\nMinimum Cost = %d\n", minCost);
    printf("Execution Time = %lf seconds\n",execution_time);
    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter Cost Adjacency Matrix:
(Enter 9999 for no edge)
9999 12 41 9999 14
12 9999 30 25 9999
41 30 9999 12 11
9999 25 12 9999 6
14 9999 11 6 9999

Edges in MST:
0 -> 1  Cost = 12
0 -> 4  Cost = 14
4 -> 3  Cost = 6
4 -> 2  Cost = 11

Minimum Cost = 43
Execution Time = 0.001000 seconds
```

Graph:

Time Complexity : O(n^3)



b)

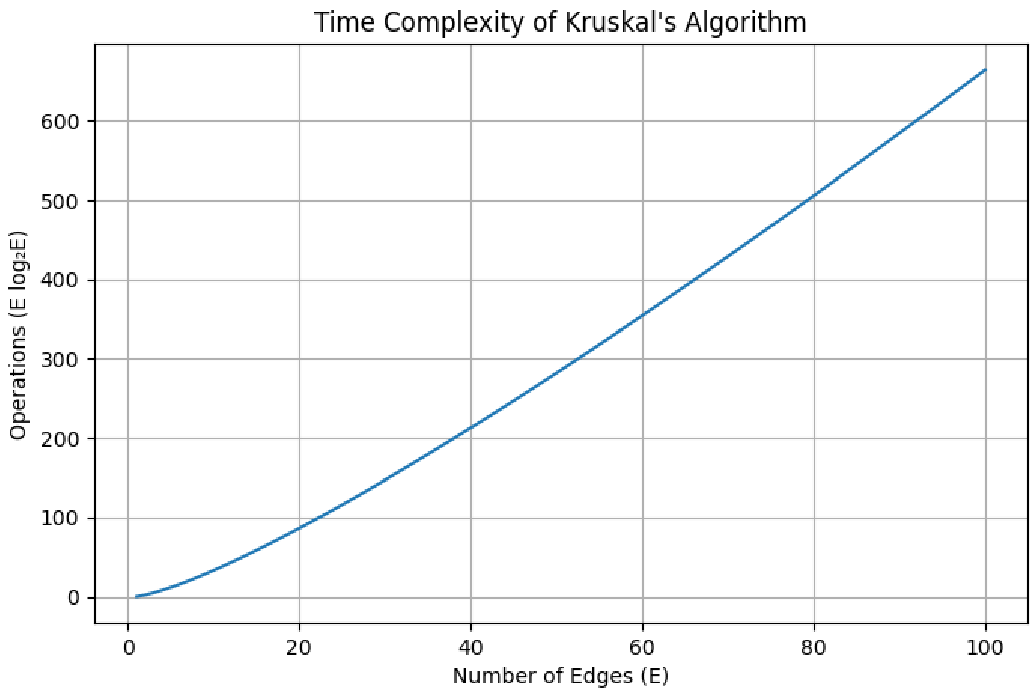
```c
#include <stdio.h>
#include <time.h>
#define MAX 100
int parent[MAX];
int find(int v)
{
    while(parent[v] != 0)
        v = parent[v];
    return v;
}
void unite(int u, int v)
{
    parent[v] = u;
}
int main()
{
    int n, e;
    int u[MAX], v[MAX], w[MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &e);
    printf("\nEnter edges (source destination weight):\n");
    for(int i = 0; i < e; i++)
    {
        scanf("%d %d %d",
              &u[i],
              &v[i],
              &w[i]);
    }
    clock_t start, end;
    double execution_time;
    start = clock();
    for(int i = 0; i < e - 1; i++)
    {
        for(int j = 0; j < e - i - 1; j++)
        {
            if(w[j] > w[j + 1])
            {
                int temp;
                temp = w[j];
                w[j] = w[j + 1];
                w[j + 1] = temp;
                temp = u[j];
                u[j] = u[j + 1];
                u[j + 1] = temp;
                temp = v[j];
                v[j] = v[j + 1];
                v[j + 1] = temp;
            }
        }
    }
    int minCost = 0;
    int count = 0;
    printf("\nEdges in MST:\n");
    for(int i = 0; i < e && count < n - 1; i++)
    {
        int p1 = find(u[i]);
        int p2 = find(v[i]);
        if(p1 != p2)
        {
            printf("%d -- %d  Cost = %d\n",
                   u[i], v[i], w[i]);
            minCost += w[i];
            unite(p1, p2);
            count++;
        }
    }
    end = clock();
    execution_time =(double)(end - start) / CLOCKS_PER_SEC;
    printf("\nMinimum Cost = %d\n", minCost);
    printf("Execution Time = %lf seconds\n",execution_time);
```

Output:

```
Enter number of vertices: 4
Enter number of edges: 5

Enter edges (source destination weight):
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in MST:
2 -- 3  Cost = 4
0 -- 3  Cost = 5
0 -- 2  Cost = 6

Minimum Cost = 15
Execution Time = 0.001000 seconds
```

Graph:

Time Complexity : O(n log n)



Lab Program 9:

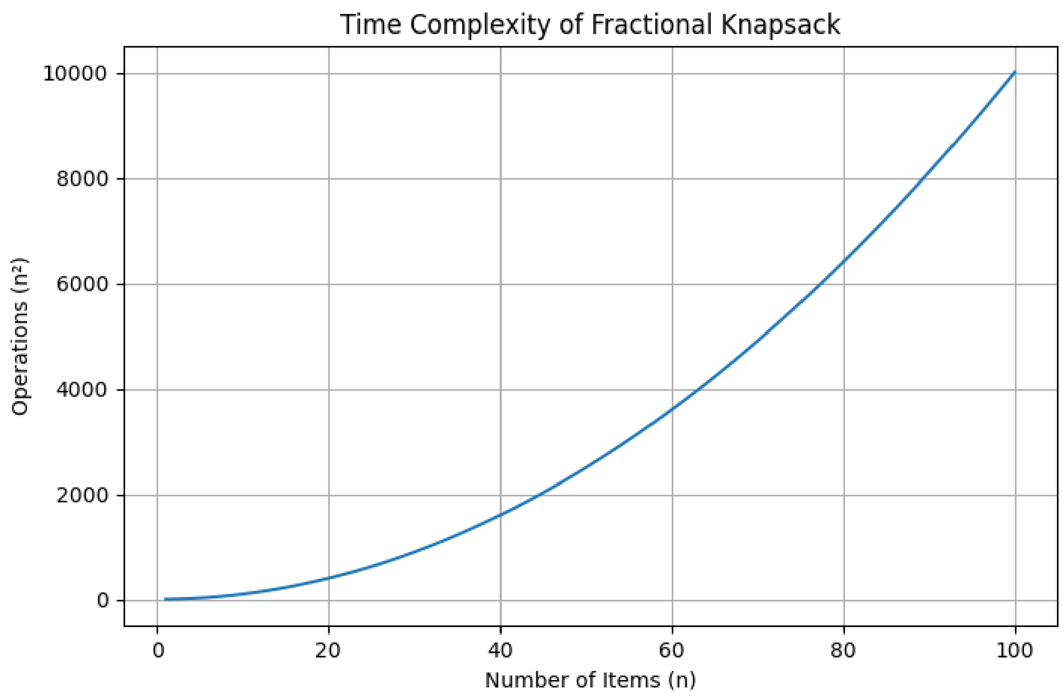
```c
#include <stdio.h>
#include <time.h>
int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int value[n], weight[n];
    float ratio[n];
    printf("Enter value and weight:\n");
    for(int i = 0; i < n; i++) {
        scanf("%d %d", &value[i], &weight[i]);
        ratio[i] = (float)value[i] / weight[i];
    }
    printf("Enter capacity: ");
    scanf("%d", &W);
    clock_t start = clock();
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(ratio[j] < ratio[j + 1]) {
                float temp = ratio[j];
                ratio[j] = ratio[j + 1];
                ratio[j + 1] = temp;
                int t1 = value[j];
                value[j] = value[j + 1];
                value[j + 1] = t1;
                int t2 = weight[j];
                weight[j] = weight[j + 1];
                weight[j + 1] = t2;
            }
        }
    }
    float totalProfit = 0.0;
    for(int i = 0; i < n; i++) {
        if(weight[i] <= W) {
            totalProfit += value[i];
            W -= weight[i];
        } else {
            totalProfit += value[i] * ((float)W / weight[i]);
            break;
        }
    }
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Maximum Profit = %.2f\n", totalProfit);
    printf("Execution Time = %f seconds\n", time_taken);
    return 0;
}
```

Output:

```
Enter number of items: 5
Enter value and weight:
200 5
500 10
300 1
600 4
400 3
Enter capacity: 6
Maximum Profit = 1033.33
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : O(n^2)



Lab Program 10:

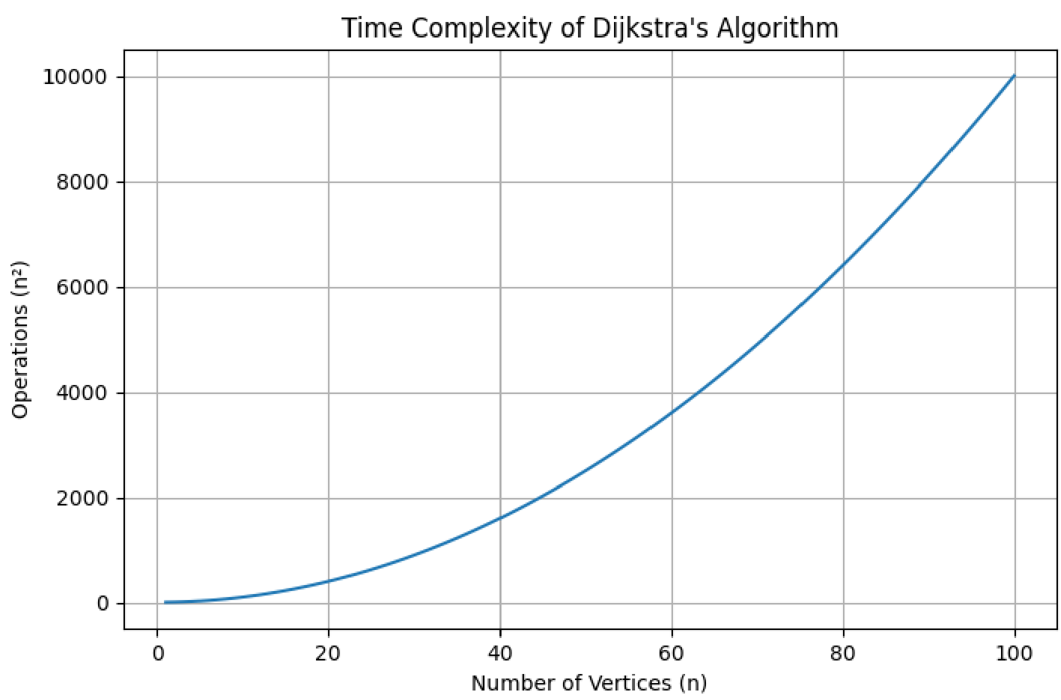
```c
#include <stdio.h>
#include <time.h>
#define MAX 100
#define INF 99999
int minDistance(int dist[], int visited[], int n) {
    int min = INF, min_index;
    for (int i = 0; i < n; i++) {
        if (visited[i] == 0 && dist[i] <= min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}
void dijkstra(int graph[MAX][MAX], int n, int src) {
    int dist[MAX], visited[MAX];
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
    dist[src] = 0;
    clock_t start = clock();
    for (int count = 0; count < n - 1; count++) {
        int u = minDistance(dist, visited, n);
        visited[u] = 1;
        for (int v = 0; v < n; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Vertex\tDistance from Source\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\n", i, dist[i]);
    }
    printf("Execution Time = %f seconds\n", time_taken);
}
int main() {
    int n, graph[MAX][MAX], src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
        }
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    dijkstra(graph, n, src);
    return 0;
}
```

Output:

```
Enter number of vertices: 4
Enter adjacency matrix:
0 10 0 30
10 0 50 0
0 50 0 20
30 0 20 0
Enter source vertex: 1
Vertex  Distance from Source
0       10
1       0
2       50
3       40
Execution Time = 0.000000 seconds
```

Graph:

Time Complexity : O(n^2)



Lab Program 11:

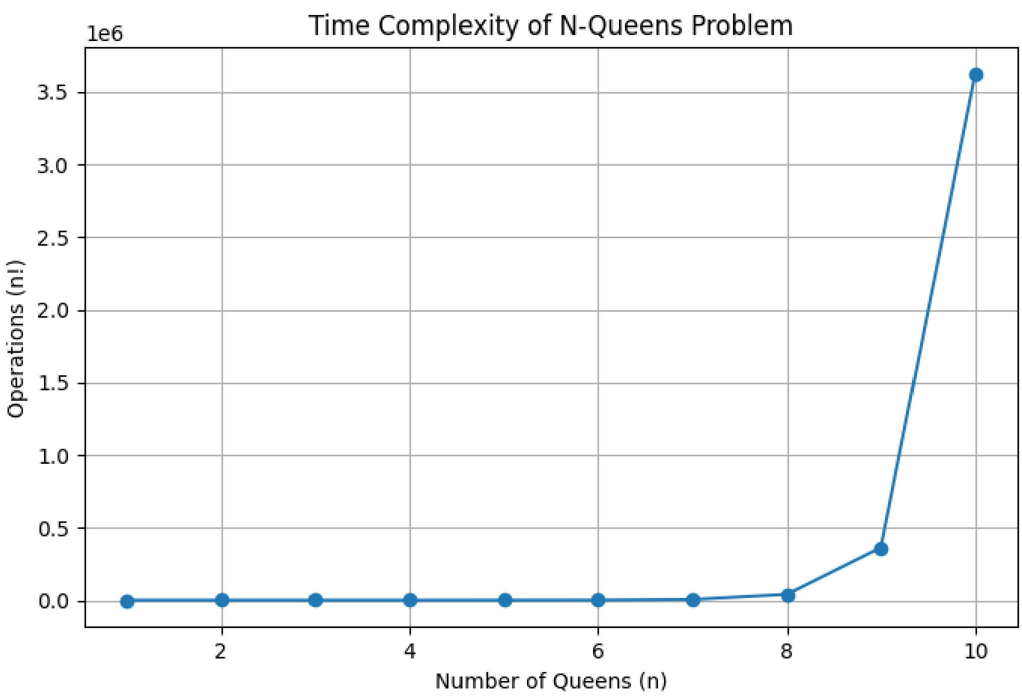
```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 20
int board[MAX];
int solutionCount = 0;
int isSafe(int row, int col)
{
    int i;
    for (i = 0; i < row; i++)
    {
        if (board[i] == col)
            return 0;
        if (abs(board[i] - col) == abs(i - row))
            return 0;
    }
    return 1;
}
void printBoard(int n)
{
    int i, j;
    printf("\nSolution %d:\n\n", ++solutionCount);
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (board[i] == j)
                printf(" Q ");
            else
                printf(" . ");
        }
        printf("\n");
    }
}
void solveNQueens(int row, int n)
{
    int col;
    if (row == n)
    {
        printBoard(n);
        return;
    }
    for (col = 0; col < n; col++)
    {
        if (isSafe(row, col))
        {
            board[row] = col;
            solveNQueens(row + 1, n);
        }
    }
}
int main()
{
    int n;
    clock_t start, end;
    double cpu_time_used;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    start = clock();
    solveNQueens(0, n);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("\nTotal Solutions = %d\n", solutionCount);
    printf("Execution Time = %f seconds\n", cpu_time_used);
    return 0;
}
```

Output:

```
Enter number of queens: 4

Solution 1:

 .  Q  .  .
 .  .  .  Q
 Q  .  .  .
 .  .  Q  .

Solution 2:

 .  .  Q  .
 Q  .  .  .
 .  .  .  Q
 .  Q  .  .

Total Solutions = 2
Execution Time = 0.004000 seconds
```

Graph:

Time Complexity : O(n!)



LeetCode Problems:

11. Container with most water

Code

C Auto

```c
int maxArea(int* height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int max = 0;
    while (left < right) {
        int width = right - left;
        int h;
        if (height[left] < height[right]) {
            h = height[left];
            left++;
        } else {
            h = height[right];
            right--;
        }int area = h * width;
        if (area > max) {
            max = area;
        }
    }
    printf("%d", max);
    return max;
}
```

Saved Ln 21, Col 2

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

34. Find first and last position of element in sorted array

Code

C Auto

```c
int* searchRange(int* nums, int numsSize, int target, int* returnSize)
{
    int *retArr= (int *) malloc (2*sizeof(int));
    *returnSize=2;
    retArr[0]=-1;
    retArr[1]=-1;
    for(int i=0;i<numsSize;i++)
    {
        if(nums[i]==target)
        {
            retArr[0]=i;
            while(i+1<numsSize&&nums[i+1]==target)
            {
                i++;
            }
            retArr[1]=i;
            break;
        }
    }
    return retArr;
}
```

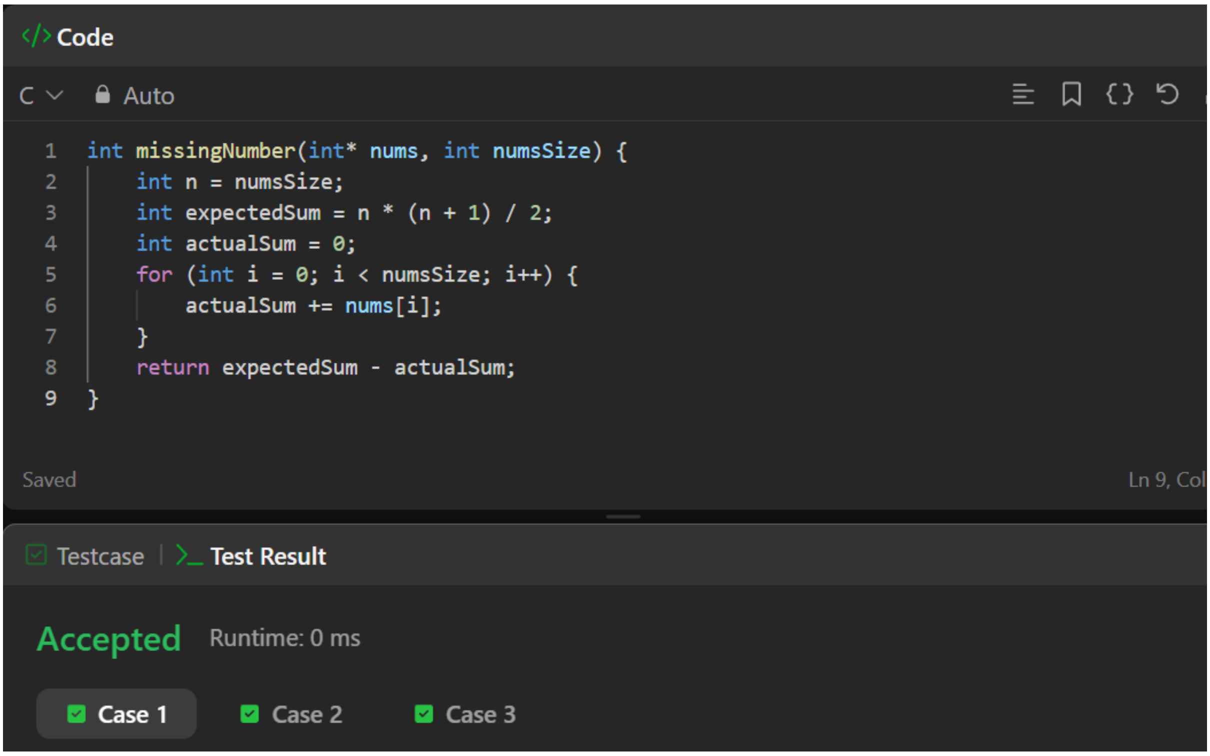
Saved Ln 21, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

268. Missing number




```c
int missingNumber(int* nums, int numsSize) {
    int n = numsSize;
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for (int i = 0; i < numsSize; i++) {
        actualSum += nums[i];
    }
    return expectedSum - actualSum;
}
```

1636. Sort array by increasing frequency

Code

C Auto

```c
int freq[201];
int compare(const void *a, const void *b)
{
    int x = *(int *)a;
    int y = *(int *)b;
    if (freq[x + 100] != freq[y + 100])
        return freq[x + 100] - freq[y + 100];
    return y - x;
}
int* frequencySort(int* nums, int numsSize, int* returnSize)
{
    for (int i = 0; i < 201; i++)
        freq[i] = 0;
    for (int i = 0; i < numsSize; i++)
        freq[nums[i] + 100]++;
    qsort(nums, numsSize, sizeof(int), compare);
    *returnSize = numsSize;
    return nums;
}
```

Saved Ln 17, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

207. Course Schedule

Code

C Auto

```c
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize, int* prerequisitesColSize) {
    int indegree[numCourses];
    int graph[numCourses][numCourses];
    int graphSize[numCourses];
    for(int i = 0; i < numCourses; i++) {
        indegree[i] = 0;
        graphSize[i] = 0;
    }
    for(int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][graphSize[b]++] = a;  // b → a
        indegree[a]++;
    }
    int queue[numCourses];
    int front = 0, rear = 0;
    for(int i = 0; i < numCourses; i++) {
        if(indegree[i] == 0)
            queue[rear++] = i;
    }
    int count = 0;
    while(front < rear) {
        int course = queue[front++];
        count++;
        for(int i = 0; i < graphSize[course]; i++) {
            int next = graph[course][i];
            indegree[next]--;
            if(indegree[next] == 0)
                queue[rear++] = next;
        }
    }
    return count == numCourses;
}
```

Saved Ln 33, Col

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

801. Minimum swaps to make sequences increasing

Code

C Auto

```c
int minSwap(int* nums1, int nums1Size, int* nums2, int nums2Size) {
    int keep = 0;   // no swap at i
    int swap = 1;   // swap at i (1 swap at index 0)
    for (int i = 1; i < nums1Size; i++) {
        int newKeep = 1e9;
        int newSwap = 1e9;
        // Case 1: sequences already increasing without swap
        if (nums1[i] > nums1[i-1] && nums2[i] > nums2[i-1]) {
            newKeep = keep;          // no swap needed
            newSwap = swap + 1;      // swap continues
        }
        // Case 2: cross swap condition
        if (nums1[i] > nums2[i-1] && nums2[i] > nums1[i-1]) {
            newKeep = fmin(newKeep, swap);        // swap before, not now
            newSwap = fmin(newSwap, keep + 1);    // no swap before, swap now
        }
        keep = newKeep;
        swap = newSwap;
    }
    return fmin(keep, swap);
}
```

Saved Ln 21, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

287. Find the duplicate number

Code

C Auto

```c
int findDuplicate(int* nums, int n) {
    int* seen = calloc(n, sizeof(int));
    for (int i = 0; i < n; i++) {
        if (seen[nums[i]]) {
            free(seen);
            return nums[i];
        }
        seen[nums[i]] = 1;
    }
    free(seen);
    return -1;
}
```

Saved Ln 1, Col

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3